

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using PyTorch.

2 Learning Outcomes

Students will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n) \quad (1)$$

where

- X: Input Image
- K: Kernel (Filter)
- Y: Feature Map

The output feature map size is:

$$\frac{N - F + 2P}{S} + 1 \quad (2)$$

where

- N: Input Size
- F: Filter Size
- P: Padding
- S: Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} 1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4 Numerical Example 1: Convolution

Consider the following input image and kernel.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position: $1(1) + 2(0) + 4(0) + 5(1) = 6$

Second position: $2(1) + 3(0) + 5(0) + 6(1) = 8$

Third position: $4(1) + 5(0) + 7(0) + 8(1) = 12$

Fourth position: $5(1) + 6(0) + 8(0) + 9(1) = 14$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5 Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,

$$\text{Window 1: } \begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

$$\text{Window 2: } \begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

$$\text{Window 3: } \begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

$$\text{Window 4: } \begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6 Numerical Example 3: Parameter Calculation

Suppose

- Input Image: $32 \times 32 \times 3$
- Convolution Layer Filters = 16
- Kernel = 3×3

Parameters

$$\begin{aligned} & (3 \times 3 \times 3 + 1) \times 16 \\ &= (27 + 1) \times 16 \\ &= 448 \end{aligned}$$

Therefore, Total trainable parameters = 448.

7 Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

8 Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare:

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare:

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare:

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Max Pool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Max Pool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Train for:

- Optimizer: Adam

- Epochs: 20
- Batch Size: 32

Task 7

Evaluate:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9 Source Code

Github Repository link: <https://github.com/Kamlesh-DevOP/deep-learning-lab/blob/main/lab-3/Lab3.ipynb>

10 Plots with Inference

1. Sample Images



Figure 1: CIFAR-10 Sample Images

Inference: The dataset consists of diverse, low-resolution (32×32) color images representing 10 distinct classes. The visual samples demonstrate significant intra-class variations in background, lighting, and object orientation, which the CNN must learn to generalize.

2. Class Distribution

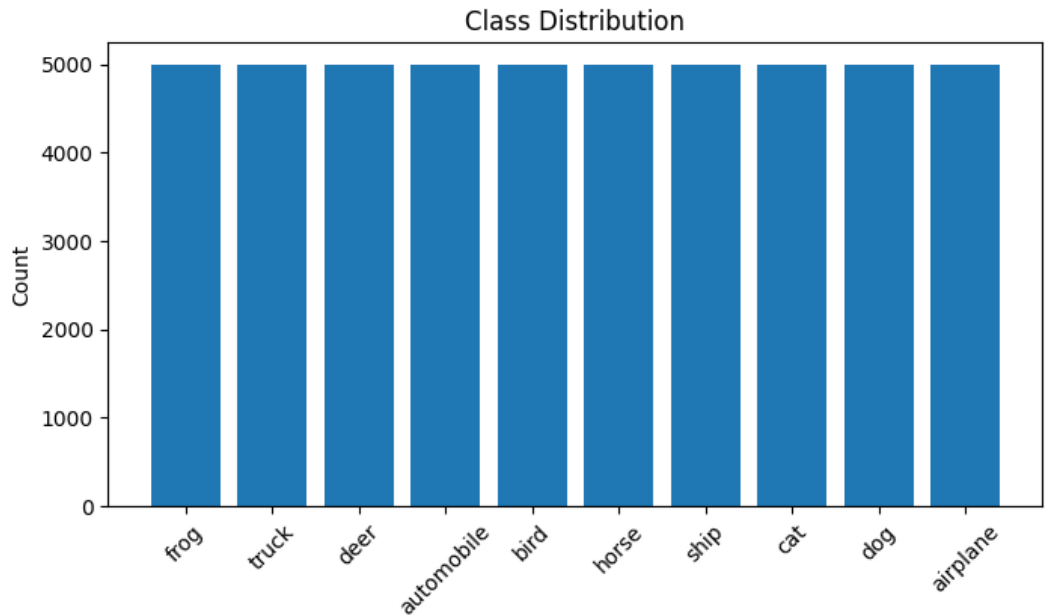


Figure 2: Training Dataset Class Distribution

Inference: The CIFAR-10 training dataset is perfectly balanced, with exactly 5,000 samples for each of the 10 classes. This uniform distribution is ideal, as it prevents the model from developing a bias toward any majority class during the training process.

3 & 4. Training and Validation Accuracy

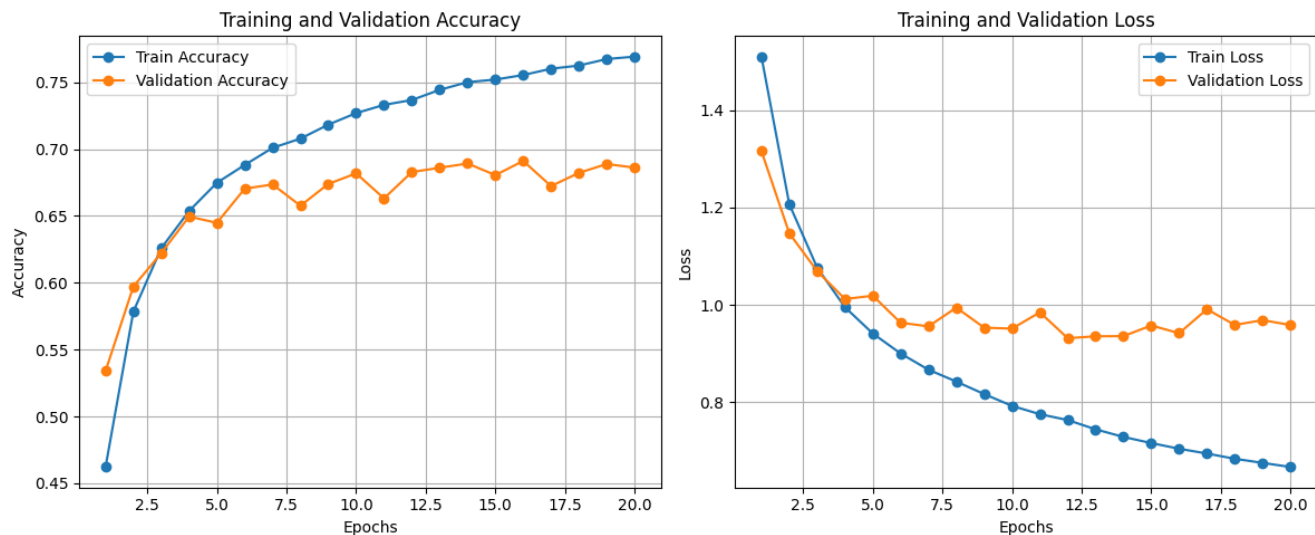


Figure 3: Accuracy and Loss Curves over 20 Epochs

Inference: The steady rise in training accuracy (reaching 77%) paired with a plateauing validation accuracy (around 68%) after epoch 10 indicates that the model is learning effectively but begins to

overfit. Introducing regularization techniques such as Dropout could help bridge this generalization gap.

5 & 6. Training and Validation Loss

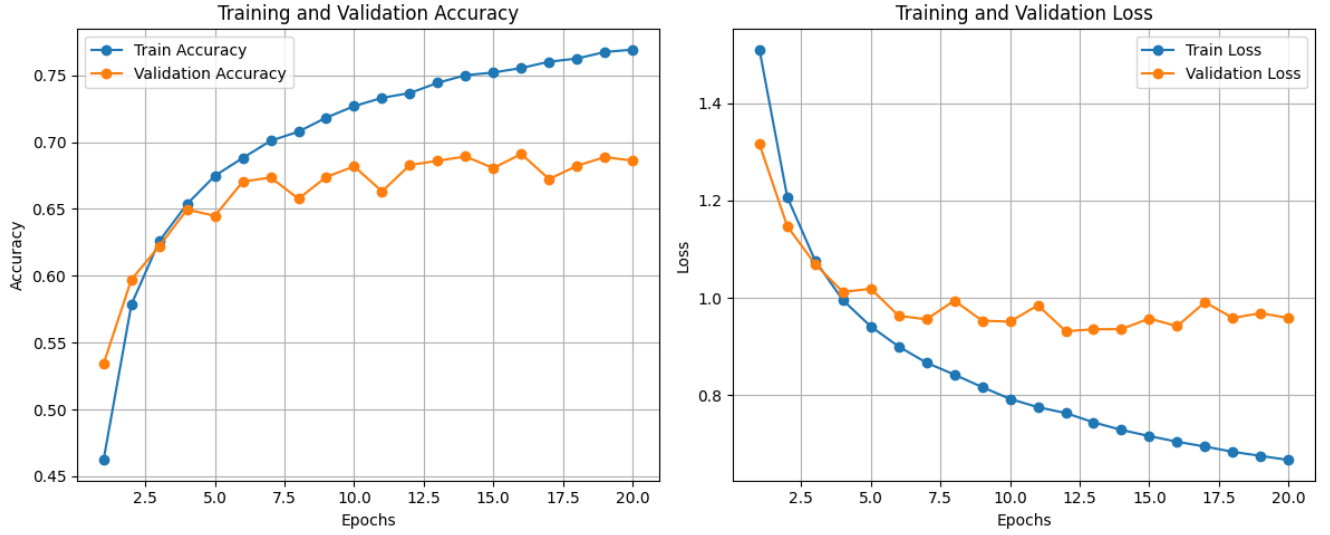


Figure 4: Accuracy and Loss Curves over 20 Epochs

Inference: The divergence between the continuously decreasing training loss and the stabilizing validation loss (which bottoms out near epoch 13) further confirms overfitting in the later epochs. Implementing an early stopping callback around epoch 10–12 would yield the most optimal network weights.

7. Feature Maps

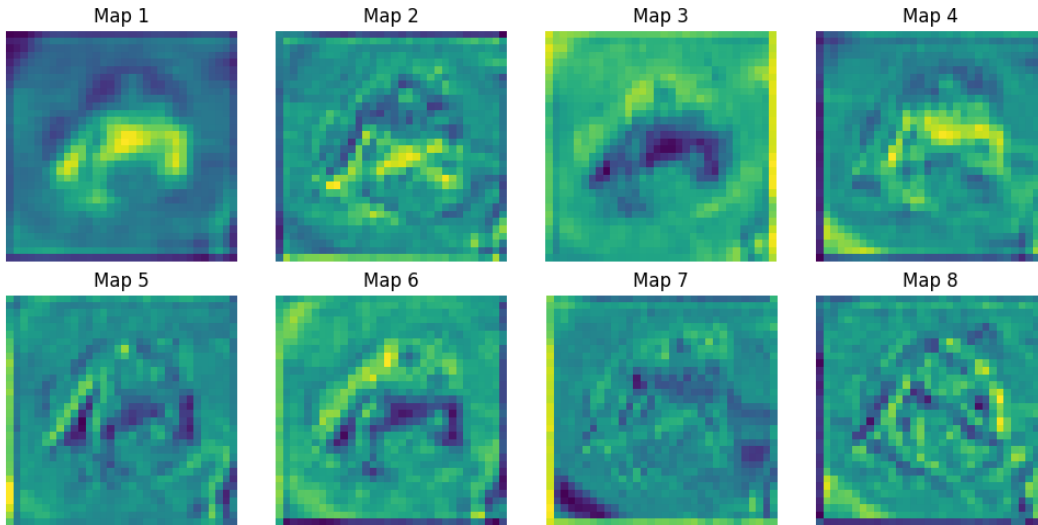


Figure 5: Visualized Feature Maps from First Convolutional Layer

Inference: The first convolutional layer successfully captures fundamental low-level spatial features, such as prominent edges, outlines, and color gradients. These individual feature maps highlight distinct

regional activations where the 3×3 filters detected specific visual patterns within the input image.

8. Confusion Matrix

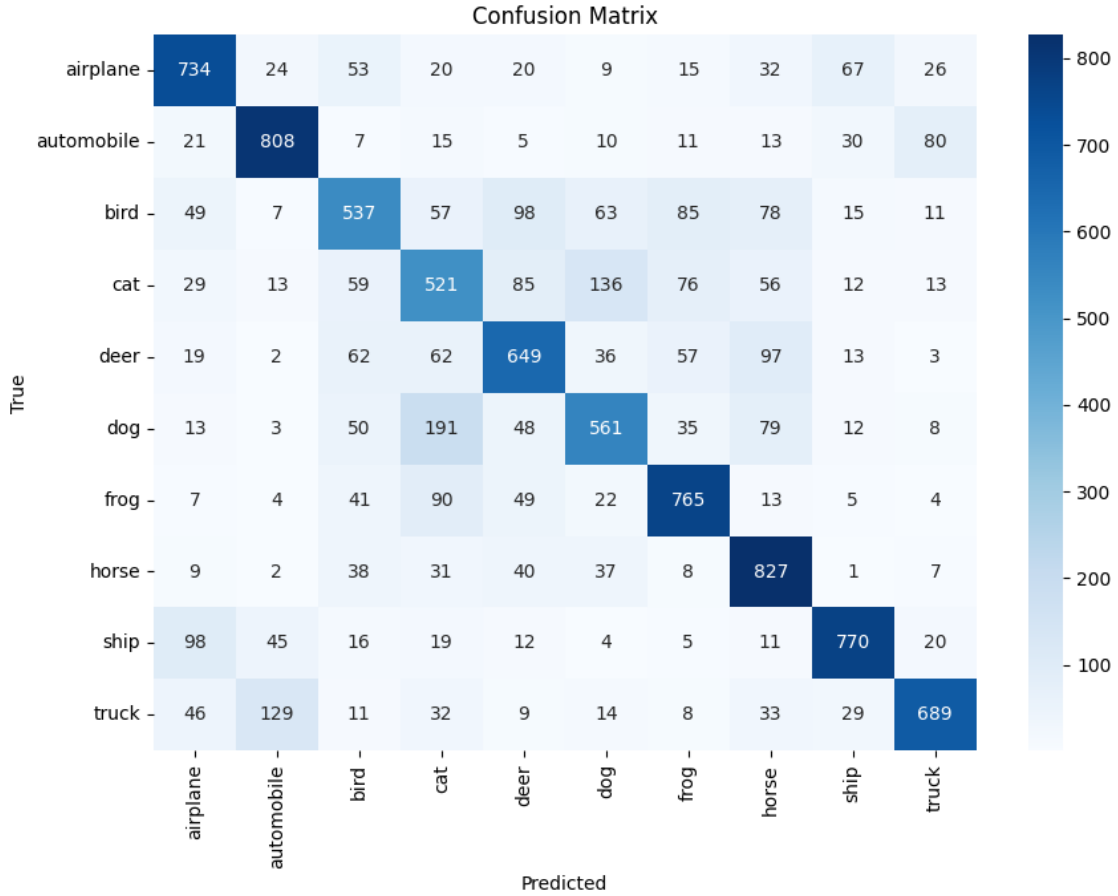


Figure 6: Model Confusion Matrix on Test Data

Inference: The model performs well overall, evidenced by the strong diagonal values representing correct predictions. However, it noticeably struggles with semantically and visually similar classes, frequently confusing cats with dogs, and automobiles with trucks.

11 Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

Using the spatial dimension formula for convolutional layers:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

$$O = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1$$

$$O = \lfloor 31.5 \rfloor + 1 = 31 + 1 = 32$$

The resulting output feature map size is **32×32** .

2. **Compute the trainable parameters of a convolution layer having 64 filters of size 3x3 and an RGB input.**

Using the parameter calculation formula (where $C_{in} = 3$ for RGB):

$$P = ((F \times F \times C_{in}) + 1) \times C_{out}$$

$$P = ((3 \times 3 \times 3) + 1) \times 64$$

$$P = (27 + 1) \times 64 = 28 \times 64 = 1792$$

Total trainable parameters = **1,792**.

3. **Compare ReLU and Sigmoid activation functions.**

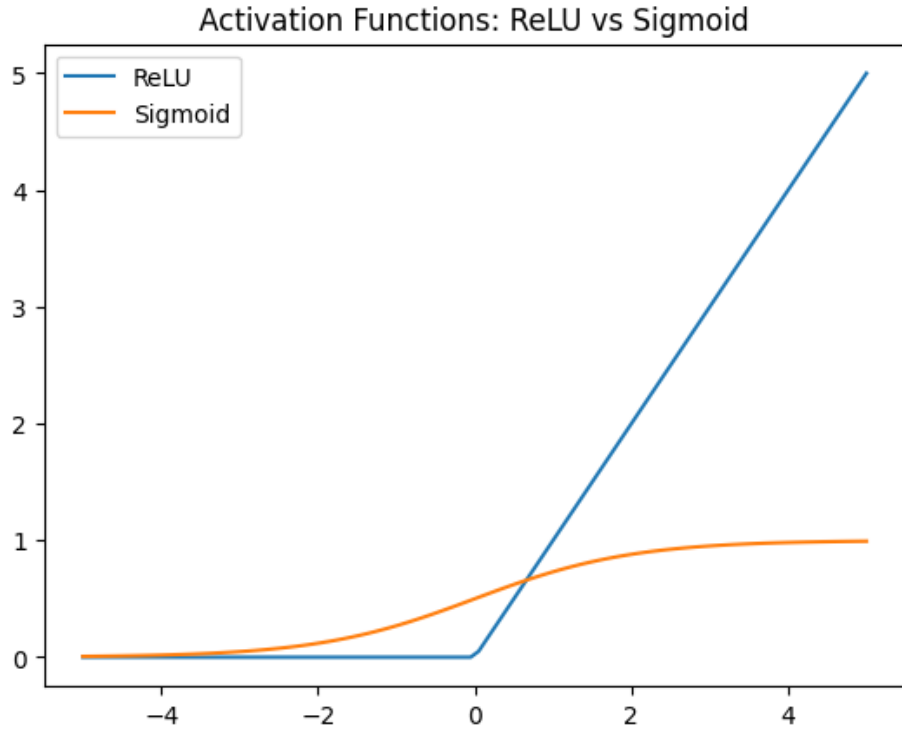


Figure 7: ReLU vs Sigmoid Activation

ReLU (Rectified Linear Unit): Defined mathematically as $f(x) = \max(0, x)$. It is computationally efficient and effectively mitigates the vanishing gradient problem for positive values, leading to much faster convergence in deep CNNs.

Sigmoid: Defined as $f(x) = \frac{1}{1+e^{-x}}$. It squashes outputs to a range strictly between 0 and 1. It is prone to the vanishing gradient problem in deep networks and is computationally more expensive due to the exponential operation.

4. **Replace Max Pooling with Average Pooling and compare the results.**

Replacing Max Pooling with Average Pooling yields the exact same spatial output dimensions (a final output shape of $[1, 10]$ for a single image pass). Conceptually, Max Pooling extracts the most prominent and intense features (such as sharp edges), which is usually optimal for object recognition tasks like CIFAR-10. Average Pooling calculates the local mean, which smooths out the features, sometimes leading to blurred representations and a slight drop in classification accuracy.

5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.**

Increasing the number of filters from 16 to 64 expands the model's capacity, allowing it to extract

and learn a much wider variety of complex visual features, which generally improves classification accuracy (provided the model is regularized to prevent overfitting). However, this architectural change drastically increases the network’s parameter count (jumping to **157,578** total parameters in our modified model). As a direct consequence, memory consumption and computation time per training epoch increase significantly.

12 Results

Overall CNN Performance

The implementation and training of the Convolutional Neural Network on the CIFAR-10 dataset yielded the following performance metrics:

- **Training Accuracy:** 0.7584
- **Testing Accuracy:** 0.6840
- **Precision:** 0.6969
- **Recall:** 0.6840
- **F1-score:** 0.6868
- **Number of Parameters:** 25,578

Pooling Comparison (Task 5)

Comparison of spatial dimensions and accuracy between Max Pooling and Average Pooling (evaluated on a baseline model over 3 epochs):

- **Max Pool Output Size:** $1 \times 16 \times 16 \times 16$
- **Avg Pool Output Size:** $1 \times 16 \times 16 \times 16$
- **Max Pool Accuracy:** 0.5938 (59.38%)
- **Average Pool Accuracy:** 0.5441 (54.41%)

13 Discussion

Answer the following:

1. Why is convolution preferred over fully connected layers for images?

Convolutional layers use parameter sharing and sparse connectivity, which drastically reduces the number of weights required. More importantly, they preserve the 2D spatial structure of an image, whereas fully connected layers (MLPs) flatten the input and inherently lose important spatial relationships between neighboring pixels.

2. How does stride affect the feature map size?

The stride determines how many pixels the filter shifts across the input image at each step. Increasing the stride directly downsamples the output, resulting in a significantly smaller spatial feature map size.

3. What is the role of padding?

Padding involves adding extra pixels (typically zeros, known as zero-padding) around the border of the input image. It is used to prevent the spatial dimensions from shrinking too quickly after passing through multiple convolutional layers and ensures that the features at the very edges of the input image are properly captured and processed.

4. Why is pooling used?

Pooling (such as Max or Average Pooling) is used to progressively reduce the spatial size of the feature maps. This reduces the computational load, decreases the number of parameters to help control overfitting, and provides a degree of spatial translation invariance (meaning the network can still recognize a feature even if it shifts slightly in the input image).

5. **How do feature maps represent image characteristics?**

Each feature map is the output of a specific filter sliding across the input. Because the network learns and optimizes the filter weights during training, each feature map effectively becomes a specialized feature detector. Early layers typically highlight basic characteristics like edges and colors, while deeper layers capture complex shapes and high-level object textures.

6. **Why do CNNs require fewer parameters than MLPs?**

CNNs require significantly fewer parameters due to two main principles: **weight sharing** (the exact same filter weight matrix is applied across every spatial location of the input) and **sparse connectivity** (each output neuron only connects to a small, localized region of the input). In contrast, an MLP connects every single input pixel to every single neuron in the subsequent layer, which leads to a massive explosion in the number of parameters for high-dimensional image data.

14 References

1. Goodfellow et al., Deep Learning.
2. Bishop, Pattern Recognition and Machine Learning.
3. Haykin, Neural Networks and Learning Machines.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.